

Metagraph Encodings over Identified Triples: Exact Images, Temporal Integrity, and Structural Depth

Volodymyr Pavlyshyn

Independent researcher

pavlyshyn@tiramemsu.com

Revision 2, 5 October 2026

Abstract

Identified triples give statements identities that other statements can reference, so set-valued incidence, higher-order endpoints and containment fit on one carrier. We argue that this representation is cheap and that invertibility is a separate integrity obligation, and we state that obligation exactly. For finite directed hypergraphs, nonempty-end Basu–Blanning metagraphs, identified higher-order metagraphs, a well-founded ubergraph fragment and a finite attributed annotating presentation, we give source signatures and exact image constraints under which encoding and decoding are inverse up to renaming. The roundtrip proofs are short bookkeeping; the contribution is the constraint set, and we show that each constraint is necessary. Four results go beyond roundtrips. A temporal theorem separates asserted rows from retained referents, so an annotation may be valid outside its bearer’s interval. A supersession relation records identity across versions. An executable-metapath recognizer decides in linear time whether an edge set can fire, and we characterize when it agrees with walk reachability. Reference rank gets operational consumers: the number of strata for referentially complete bulk loading and the depth of retraction cascades. Evidence is exhaustive bounded enumeration, seeded randomized property tests, executable SHACL validators and a Lean proof of the directed roundtrip. We report no performance or retrieval results; scope limits are collected in Section 11.

1 Introduction

An episode may group several facts, a causal relation may connect events that are themselves relations, and an assessment may concern whether a fact belongs to an episode. Representing these structures in a database requires decisions about identity, incidence, containment, provenance and time. The question addressed here is not whether a relational or graph database can simulate them. It is which constraints make a common statement-identity representation invertible, and which properties survive the choice of representation.

An *identified triple* is a row (e, s, p, o) whose identifier e can appear as an endpoint of another row. This primitive predates the present work: closely related models include MillenniumDB domain graphs, multilayer graphs and statement graphs [1, 12, 37], and in RDF 1.2 an identified triple is a triple together with a reifier [16]. Our earlier layered bitemporal graph calculus adds statement lifetimes and valid time [26]. Here we study metagraph representations on the untimed carrier and state the conditions for temporal lifting. The informal metagraph architecture of *Metagraph for AI Agents* [27] motivates the application but is not evidence for the formal results.

Thesis. Uniform statement identity is cheap: every encoding below is one row per source clause. Invertibility is a different matter. A store in a permissive row language can contain shapes that no source structure produces, and the question “which stores decode, and to what” needs a constraint set that is global (extensionality, acyclicity, grouped cardinalities) and not

merely typed rows. We therefore distinguish a *row language*, which only permits shapes, from an *exact image*, whose constraints ensure that decoding recovers a specified source. Most of the work, and nearly all of the room for error, is in writing the images exactly; the roundtrip theorems that follow are bookkeeping once this is done, and we say so rather than presenting them as depth. Section 4 proves that every constraint is necessary by exhibiting, for each, a store that violates only that constraint.

The paper makes five contributions; the first is the foundation for the others.

1. Explicit source signatures, encoders, decoders and exact image classes for four families of finite structures, with a formal isomorphism notion and a necessity witness for every image constraint (Sections 3 and 4).
2. A snapshot theorem with two inclusion policies (asserted rows, and asserted rows plus retained referents) that lets epistemic annotations be valid outside their bearer’s interval, a statement of why extensional images are fragile under time, and a supersession relation with an integrity condition for identity across versions (Section 6).
3. Executable metapaths: linear-time recognition of an edge set that can fire under all-inputs-required semantics, and a characterization of when this agrees with walk reachability (Section 7).
4. Operational consumers for reference rank, namely bulk-load strata and retraction-cascade depth, with representation consequences for anchored and compact encodings (Section 8).
5. Validation by bounded exhaustive enumeration, seeded randomized property tests, SHACL validators and a Lean proof of the directed roundtrip (Section 10).

2 Related Models and Scope

Metagraphs. Basu–Blanning metagraphs have a generating set X and edges that are pairs $\langle V_e, W_e \rangle$ of subsets of X [4, 5]. In the presentation of Parsonage et al. [25] (their Definition 2.2) the invertex and outvertex may be empty and are not required to be disjoint; we restrict the main directed encoding to nonempty ends and impose no disjointness constraint, so a vertex may be both an input and an output of an edge. Allowing empty ends means dropping the corresponding cardinality constraints, not changing the row representation. Their distinction from directed hypergraphs concerns the associated calculus as well as terminology [9, 25]. Joslyn and Nowak’s ubergraphs extend set-valued incidence to recursively nested edges [19]; our member-role construction is an identified-statement realization of that incidence idea. The annotating tradition distinguishes vertices, metaverices, edges and metaedges, with metaedges carrying fragments [6, 10]; our attributed encoding preserves this data structure and does not implement function or rule agents. Nested-graph models such as higraphs, bigraphs and hypernodes make other containment choices [15, 23, 28]; the hypernode model in particular is a direct precedent for well-founded nesting, and we do not impose a forest or region semantics.

Anchor-plus-role encodings have a long history. The pattern of an intermediate node with one property per participant is the W3C n -ary relation pattern [24]. TypeDB’s relations take typed role players, and relations may themselves play roles in other relations [35]; this is the closest production analogue of the in/out/member encoding. Hyper-relational knowledge graphs attach qualifier pairs to statements [8], which is the shape of an annotation row on an identified statement. OpenCog’s AtomSpace and HyperGraphDB admit links whose targets are themselves links [14, 18]. Hernández et al. compare reification schemes empirically on Wikidata [17]; our row counts are formulas over a chosen schema, not measurements of any store. In RDF 1.2 terms, an identified triple (e, s, p, o) is a reifier e with e `rdf:reifies` $\langle\langle s p o \rangle\rangle$ [16]. Neither the carrier nor the anchor-plus-role pattern is new here.

Source convention	Structure retained	Exact-image obligation here
Basu–Blanning	Set-valued directed ends	Nonempty ends; unique endpoint pair
Identified higher-order	Independent edge identity; edge endpoints	Role types; no edge as its own end
Well-founded ubergraph fragment	Recursive extensional sets	Member uniqueness; acyclicity; extensionality
Attributed annotating presentation	Four classes; fragments; direction	Class markers; unique ends and direction

Table 1: Representation contracts, not a comparison of native and encoded expressiveness. The source restrictions are part of each result.

Statement images. Gelling, Fletcher and Schmidt define statement graphs as acyclic graphs with subject/predicate/object roles and define images for RDF, RDF-star and property graphs with bidirectional mappings [11, 12]. For our restricted endpoint types, replacing each row by a statement vertex with three role arcs gives the corresponding statement-graph view; this is a representation correspondence, not an expressiveness separation. Chernenkiy et al. already study relational, document-oriented and graph storage mappings, including nested assertions by statement identifier [7]. Meta-Property Graphs make properties and labels identifiable and associate nodes with well-founded sets of graph objects [30]; their Definition 4.1 does not require a reified edge’s endpoints to lie in the same substructure, so containment need not mean an induced subgraph, which matches the explicit membership semantics used here. Our focus is metagraph-specific image constraints and their depth, containment and temporal contracts.

Temporal integrity. The interval-containment condition of Section 6 is the classical *sequenced temporal referential integrity* constraint of bitemporal databases: a child’s period must be covered by its parent’s [22, 32]. We name it as such and then argue that, for epistemic annotations, it is the wrong default.

Recent metagraph work. Tarassov, Kaganov and Gapanyuk’s publisher abstract describes active metagraphs, a calculus and granulation [33]; Vinnikov, Nardid and Gapanyuk emphasize hierarchical operations and a binary nesting relation [36]; the authors’ conference abstract for the incidence/nesting work describes matrix-based transformations [20], with the journal record [21]. These comparisons are limited to accessible abstracts and records, not full-text theorem comparisons, and we claim no priority over them.

3 Carrier, Isomorphism and Row Fragments

Fix pairwise disjoint sets of nodes $\mathcal{N}$, literals $\mathcal{L}$ and statement identifiers $\mathcal{I}$, with predicates $\mathcal{P} \subseteq \mathcal{N}$. Requiring $\mathcal{P} \subseteq \mathcal{N}$ excludes statement-valued predicates, unlike models in which edge labels are themselves identifiable objects [30].

Definition 1 (Carrier). A store consists of a finite $S \subseteq \mathcal{I}$ and a function

$$\kappa : S \longrightarrow (\mathcal{N} \cup S) \times \mathcal{P} \times (\mathcal{N} \cup \mathcal{L} \cup S), \quad \kappa(e) = (s_e, p_e, o_e).$$

Write $e \rightarrow d$ when $d \in S$ is an endpoint of e . The relation $\rightarrow$ is required to be acyclic, which in particular excludes $e \in \{s_e, o_e\}$.

All statement endpoints exist in S . Equal contents may have different identifiers; an identifier denotes one occurrence, not an equivalence class of equal triples. Acyclicity holds for immutable rows inserted only after their referents exist; for arbitrary imported stores it is an assumption.

Definition 2 (Rank). $\text{rk}(e) = 0$ for rows with no statement endpoints, and $\text{rk}(e) = 1 + \max_{e \rightarrow d} \text{rk}(d)$ otherwise. Set $\Lambda_k = \{e : \text{rk}(e) = k\}$.

Lemma 1 (Grounded prefixes). *Each $\Lambda_{\leq k}$ is closed under references, and every rank- k row with $k > 0$ references at least one rank- $(k-1)$ row.*

Proof. Every reference lowers rank, and the defining maximum is attained in a finite store. $\square$

3.1 Anchor convention and isomorphism

For an atom x choose a fresh node n_x and identifier $h(x)$ and set $\kappa(h(x)) = (n_x, \text{rdf:type}, \text{mg:C})$, where C is Vertex, Edge, MetaVertex or MetaEdge. Anchor nodes are pairwise distinct and occur in no other row. Reserve `rdf:type`, `mg:in`, `mg:out`, `mg:member`, `mg:src`, `mg:tgt`, `mg:directed`, `mg:container`, `mg:supersedes` and `sys:inGraph`; attribute keys lie outside this set. We write (s, p, o) for the content of a row, never as its identifier.

Definition 3 (Isomorphism). Let $F_G \subseteq \mathcal{N}$ be the set of anchor (fresh) nodes of a store G . Stores G, G' are *isomorphic*, $G \cong G'$, if there are bijections $\varphi : S \rightarrow S'$ and $\psi : F_G \rightarrow F_{G'}$ such that, writing $\bar{\varphi}$ for $\varphi \cup \psi$ extended by the identity on literals, reserved vocabulary and all non-fresh nodes, $\kappa'(\varphi(e)) = \bar{\varphi}(\kappa(e))$ for every $e \in S$ (applied componentwise). For source models, $M \cong M'$ means a bijection of atoms that preserves class markers, role incidence, direction flags and attribute pairs, and fixes attribute keys and values.

Every roundtrip statement below is an instance of this definition: $\text{dec}(\text{enc}(M)) \cong M$ and $\text{enc}(\text{dec}(G)) \cong G$ on the image.

Definition 4 (Row fragments). All fragments obey the carrier and anchor conventions and impose only shape restrictions; the exact images of Section 4 add cardinality and uniqueness. A *container* is an anchor of class MetaVertex or MetaEdge, or an anchor carrying a marker row $(h(c), \text{mg:container}, \text{true})$.

- **Row₁**: Vertex and Edge anchors and rows $(h(e), r, h(v))$, with e an Edge, v a Vertex and $r \in \{\text{mg:in}, \text{mg:out}\}$.
- **Row₂**: Row₁ with in/out objects also allowed to be Edge anchors, and direct relations $(h(x), p, h(y))$ between anchors for non-reserved p .
- **Row₃**: Row₂ plus MetaVertex and MetaEdge anchors, `src/tgt` roles from Edge or MetaEdge to Vertex or MetaVertex, `member` roles from Edge to any anchor, ground facts, container markers, and memberships $(y, \text{sys:inGraph}, h(c))$ with y an anchor or ground fact and $h(c)$ a container.
- **Row₄**: Row₃ plus direction rows on Edge or MetaEdge anchors, non-reserved property rows (y, p, d) with any row identifier y and any node, literal or row identifier d , and memberships whose subjects are any row identifiers; $\rightarrow$ stays acyclic.

Proposition 2 (Strict syntactic inclusion). $\text{Row}_1 \subsetneq \text{Row}_2 \subsetneq \text{Row}_3 \subsetneq \text{Row}_4$.

Proof. An in-role to an Edge anchor, a MetaVertex anchor, and a property row on a membership witness the three strict inclusions, respectively. $\square$

These are nested *shape classes*. We call them fragments, not normal forms: there is no normalization procedure, canonical form or rewriting into them. Section 7 discusses the one operation that maps an enriched store to an image, namely deletion of rows outside the image. The inclusion proves inclusion of permitted store shapes, not impossibility of alternative encodings into a smaller language.

4 Exact Encoding Images

A representation is *invertible on its image* if

$$\text{dec}_T(\text{enc}_T(M)) \cong M \quad (M \text{ in the source domain } \mathcal{S}_T), \quad \text{enc}_T(\text{dec}_T(G)) \cong G \quad (G \in \mathcal{J}_T).$$

This is an information-preserving statement. We do not assert categorical fullness or faithfulness without a specified class of morphisms.

Lemma 3 (Clause bijection). *Let an encoding enc_T emit, for every source clause (atom, incidence, flag or attribute), exactly one row whose content determines that clause, with anchor rows for atoms as in Section 3. Let $\mathcal{J}_T$ be the set of stores that (i) consist only of such rows, (ii) have pairwise distinct row contents, (iii) have one anchor row per anchor node, anchor nodes occurring nowhere else, and (iv) satisfy a constraint set $\mathcal{C}_T$. Define $\text{dec}_T(G)$ by reading one clause per row. Then $\text{enc}_T, \text{dec}_T$ are inverse up to isomorphism on $\mathcal{S}_T$ and $\mathcal{J}_T$ if and only if (C1) every model in $\mathcal{S}_T$ encodes into $\mathcal{J}_T$, and (C2) every store in $\mathcal{J}_T$ decodes into $\mathcal{S}_T$.*

Proof. Necessity of (C1) and (C2) is the requirement that the maps be defined on the stated domains. For sufficiency: for $M \in \mathcal{S}_T$, reading clauses back from $\text{enc}_T(M)$ gives M itself, because each row determines exactly the clause that produced it, so the atom bijection $x \mapsto$ (the atom of the anchor row at $h(x)$) is an isomorphism. For $G \in \mathcal{J}_T$, (ii) and (iii) make the map from rows to clauses injective and make anchor nodes private. Define φ by sending each anchor row to the anchor row of the same atom in $\text{enc}_T(\text{dec}_T(G))$, and each other row to the unique row of equal decoded clause; define ψ on anchor nodes accordingly. Contents correspond under $\bar{\varphi}$ by construction, and φ, ψ are bijections because every row of G yields one clause and every clause of $\text{dec}_T(G)$ comes from one row of G . $\square$

By Lemma 3 each roundtrip result below reduces to checking (C1) and (C2): the source restrictions imply the image constraints and vice versa. That comparison is the content of the definitions, and it is where mistakes occur; the proofs themselves are routine.

4.1 Directed and higher-order incidence

Definition 5 (Identified directed structures). Let X, E be disjoint finite atom sets. For each $e \in E$ choose nonempty sets $V_e, W_e \subseteq X$ for a directed hypergraph, or $V_e, W_e \subseteq (X \cup E) \setminus \{e\}$ for an identified higher-order metagraph. Distinct edge identities may have equal ends; $V_e \cap W_e$ may be nonempty.

Encode one Vertex anchor per $x \in X$, one Edge anchor per $e \in E$, one row $(h(e), \text{mg:in}, h(x))$ per $x \in V_e$ and one row $(h(e), \text{mg:out}, h(y))$ per $y \in W_e$.

Definition 6 (Directed image constraints). $\mathcal{J}_D$ consists of stores containing only Vertex/Edge anchors and in/out roles such that (D1) role subjects are Edge anchors, (D2) role objects are Vertex anchors, (D3) each edge has at least one in role, (D4) each edge has at least one out role, (D5) no extra relation, annotation, direction or attribute rows occur, (D6) no two roles have equal contents. $\mathcal{J}_{HO}$ replaces (D2) by (H1) objects are Vertex or Edge anchors, and adds (H2) no row $(h(e), r, h(e))$. $\mathcal{J}_{BB} \subseteq \mathcal{J}_D$ adds (B1) no two Edge anchors have equal in-object sets and equal out-object sets.

(B1) is extensionality and matches the finite nonempty-end Basu–Blanning source $\langle X, \{(V_e, W_e)\} \rangle$.

Proposition 4 (Directed and higher-order roundtrips). *The respective encodings are invertible onto $\mathcal{J}_D$, $\mathcal{J}_{HO}$ and $\mathcal{J}_{BB}$. Higher-order incidence may contain cycles between different edges.*

Proof. Lemma 3 applies: each clause yields one row and (D6) is the distinctness (ii). (C1): a source edge has nonempty in and out sets, giving (D3),(D4); roles go from Edge to Vertex (or Vertex/Edge in HO), giving (D1),(D2) or (H1), and the HO source restriction $V_e, W_e \not\exists e$ gives (H2); distinct (e, r, x) give distinct contents, giving (D6); no other rows are emitted, giving (D5); for BB, an extensional source has pairwise distinct pairs (V_e, W_e) , giving (B1). (C2): decoding reads V_e, W_e from role objects, nonempty by (D3),(D4), typed by (D2) or (H1) and self-free by (H2); for BB, (B1) says that turning identified edges into a set of pairs collapses none. All role rows reference only anchors, so incidence cycles between edges create no reference cycle. $\square$

4.2 A well-founded ubergraph fragment

Definition 7 (Recursive extensional incidence). An object has disjoint finite sets X, E and nonempty member sets $M_e \subseteq X \cup E$. The relation $e \rightsquigarrow a$ for $a \in M_e \cap E$ is acyclic. Distinct edges have distinct member sets. Fundamental vertices are atoms, not sets equal to edges.

This is an atom-based presentation of the nonempty well-founded fragment of the recursive set model [19]. Realize each edge as the set of its realized members. Extensionality makes realization injective on edges: if two realized sets coincide, injectivity at smaller dependency depth makes their member identities coincide, so the edges are equal; the induction terminates by acyclicity. We exclude empty edges, which the source's powerset presentation does not.

Encode anchors and one $(h(e), \text{mg:member}, h(a))$ per member. $\mathcal{J}_U$ permits only Vertex/Edge anchors and member roles, and requires (U1) member roles only, from Edge anchors, (U2) nonempty member sets, (U3) acyclicity of the decoded edge-member relation, (U4) extensionality of member sets, (U5) unique role contents.

Proposition 5 (Recursive-member roundtrips). *The member encoding is invertible onto $\mathcal{J}_U$.*

Proof. Lemma 3: (U1)–(U5) are verbatim the source restrictions on the decoded relation, giving (C1) and (C2). Acyclicity gives a finite recursive realization and (U4) prevents two edge anchors from denoting the same realized set. $\square$

(U3) and (U4) are global graph and grouped-set constraints, not row types.

4.3 An attributed annotating presentation

Definition 8 (Four-class attributed structure). Let $A = V \dot{\cup} MV \dot{\cup} E \dot{\cup} ME$ be finite. For each $e \in E \cup ME$ specify source and target in $V \cup MV$ and a boolean direction flag. For every $c \in MV \cup ME$ specify a fragment $F_c \subseteq A \setminus \{c\}$, possibly empty. Every atom has a finite set of attributes (k, d) with non-reserved key k and node or literal value d ; equal pairs occur at most once per atom.

This formalizes the four-class tuple structure of [10]. It treats identity as separate from attribute and fragment extension and permits cycles between different fragment owners; process execution and rewriting are not modelled. Encode each edge or metaedge by one src row, one tgt row and one directed row, fragment membership by $(h(y), \text{sys:inGraph}, h(c))$ and attributes by $(h(x), k, d)$. The four class markers identify containers even when fragments are empty. $\mathcal{J}_A$ requires (A1) exactly one src row per Edge/MetaEdge, (A2) exactly one tgt row, (A3) exactly one boolean directed row, (A4) src/tgt objects are Vertex or MetaVertex anchors, (A5) membership objects are MetaVertex or MetaEdge anchors, (A6) membership subjects differ from their objects, (A7) attribute keys are non-reserved, (A8) unique row contents and no other rows.

Proposition 6 (Attributed roundtrips). *The four-class encoding is invertible onto $\mathcal{J}_A \subseteq \text{Row}_4$.*

Proof. Lemma 3. Empty fragments are recovered from the owner's class marker and the absence of members, so an empty-fragment metaedge stays distinct from an ordinary edge. (A1)–(A3) recover the tuple fields, and (A5),(A6),(A8) recover fragments and attributes as sets. $\square$

Constraint	Image	Requirement	Witness violating only it
(D1)	$\mathcal{J}_D$	role subject is an Edge anchor	a role row whose subject is a Vertex anchor
(D2)	$\mathcal{J}_D$	role object is a Vertex anchor	an in-role whose object is an Edge anchor
(D3)	$\mathcal{J}_D$	each edge has an in role	an edge with only an out role
(D4)	$\mathcal{J}_D$	each edge has an out role	an edge with only an in role
(D5)	$\mathcal{J}_D$	no extra annotation, direction or attribute rows	an attribute row on a Vertex
(H2)	$\mathcal{J}_{HO}$	no edge is an end of itself	e with an in-role to e itself
(B1)	$\mathcal{J}_{BB}$	no two edges have equal in/out sets	two edges with equal in and out sets
(U1)	$\mathcal{J}_U$	only member roles	an in-role in a member structure
(U2)	$\mathcal{J}_U$	member sets are nonempty	an edge with no member
(U3)	$\mathcal{J}_U$	member relation is acyclic	e_0 a member of e_1 and e_1 of e_0
(U4)	$\mathcal{J}_U$	member sets are distinct (extensional)	two edges with the same single member
(A1)	$\mathcal{J}_A$	exactly one src per (meta)edge	a metaedge with no src row
(A2)	$\mathcal{J}_A$	exactly one tgt per (meta)edge	an edge with two tgt rows
(A3)	$\mathcal{J}_A$	exactly one direction flag	a metaedge with no direction row
(A4)	$\mathcal{J}_A$	src/tgt objects are Vertex or MetaVertex	a src role to a MetaEdge
(A5)	$\mathcal{J}_A$	membership objects are containers	a membership into a Vertex
(A6)	$\mathcal{J}_A$	membership subject differs from object	a MetaVertex member of itself
(A7)	$\mathcal{J}_A$	attribute keys are not reserved	attribute key <code>mg:in</code>

Table 2: Necessity of the image constraints. Each row is a store (or model) that violates exactly the named constraint and satisfies all others of its family. The checker verifies that the full validator rejects it and that the validator with only that constraint removed accepts it.

An extensional metavertex variant adds the requirement that no two MetaVertex anchors have equal member sets; this changes the source identity convention, not the carrier. Annotations of roles, memberships and other annotations are extensions in Row_4 and are not part of $\mathcal{J}_A$.

4.4 Every constraint is necessary

Proposition 7 (Necessity). *For each constraint in Table 2 there is a model that satisfies all other constraints of its family and violates only that one. Hence the constraint cannot be dropped without enlarging the decoded domain beyond the source signature.*

Proof. The witnesses are listed in the table. Each is verified mechanically by the retained checker, which runs the validator with the named constraint removed and checks that the witness is then accepted. $\square$

The witnesses make concrete the claim that a typed row language does not suffice: (D3) and (D4) are cardinality constraints, (B1) and (U4) are grouped-set constraints and (U3) is a graph constraint, and none is a property of a single row.

5 Containment and Provenance

A container is an anchor marked MetaVertex or MetaEdge, or an anchor with an explicit `mg:container` marker. Its members are the subjects of its `sys:inGraph` rows. An empty

container remains identifiable. Overlapping containers share members without copying rows. Two owner anchors with equal members remain distinct unless an extensional owner constraint is imposed.

Any finite directed graph, loops included, can be represented as a membership graph: create one marked Vertex container per vertex and one membership per arc. Cycles among containers reference only ground anchors, so the carrier stays acyclic, and self-membership is representable because the membership identifier differs from its owner's; it is excluded in $\mathcal{J}_A$ by (A6).

A membership has its own identity. In Row_4 it may be annotated, placed in another container and dated. In Row_3 only anchors and ground facts may be members. Transitive containment is finite reachability under $\text{sys}:\text{inGraph}^+$, even on a cyclic membership graph. Folding a chosen set inserts memberships; unfolding reads members. This implements grouping, not the quotient and morphism algebra of metagraph folding [13].

A provenance row that names a source node is descriptive. A row that references another statement participates in reference dependency. These must be distinguished when deriving retraction behavior. Under dependent-row cascade, retracting an owner removes memberships pointing to it; other members remain unless they independently reference that owner, and in the exact attributed image member anchors are ground and remain. Cascading reference deletion alone does not maintain mandatory endpoint cardinalities for surviving edge anchors.

6 Temporal Integrity

Give source components transaction lifetimes L_c and valid intervals J_c . A *component* is either an atom component, meaning an anchor with all its mandatory fields (in/out roles in a directed encoding, member roles in a recursive one, src/tgt/direction rows in the attributed one), or an optional component (a fragment membership, an attribute, or an annotation of any row). All rows of a component share its clocks. Write $c \rightsquigarrow d$ if some row of c references a row of d ; this source-level dependency relation coincides with the encoded reference relation by construction. Component c is *asserted at* (t, v) if $t \in L_c$ and $v \in J_c$.

Two inclusion policies are possible. Let $A_{t,v}$ be the set of components asserted at (t, v) and let $\text{Ret}_{t,v}$ be the least $\rightsquigarrow$ -closed set containing $A_{t,v}$.

- **Policy S (containment).** Require $L_c \subseteq L_d$ and $J_c \subseteq J_d$ for every $c \rightsquigarrow d$. The snapshot is $A_{t,v}$.
- **Policy R (retained referents).** Require only $L_c \subseteq L_d$. The snapshot is $\text{Ret}_{t,v}$: the asserted components together with every component they reference, retained filtered by transaction time alone.

The containment of Policy S is sequenced temporal referential integrity [22] and is right for mandatory structure, where an edge cannot be valid while its end is not. It is questionable for epistemic annotations. “This 2020–2021 fact was disputed in 2025” is naturally valid in 2025, outside its bearer’s interval, and a 2026 assessment of a 2020 fact likewise has its own valid time. The transaction constraint $L_c \subseteq L_d$ remains necessary in both policies: a row cannot be recorded before its referent exists.

Let $\text{Ctx}_{t,v}(H)$ be the source model whose components are $\text{Ret}_{t,v}$. It extends the asserted part of the source snapshot by retained referents, whose asserted status is read from their stored valid intervals.

Theorem 8 (Snapshot commutation). *Assume $L_c \subseteq L_d$ for every $c \rightsquigarrow d$. For every (t, v) let $\text{Snap}_{t,v}^R$ select the rows of the components in $\text{Ret}_{t,v}$. If $\text{Ctx}_{t,v}(H)$ lies in the source domain, then*

$$\text{Snap}_{t,v}^R(\text{enc}(H)) \cong \text{enc}(\text{Ctx}_{t,v}(H)), \quad \text{dec}(\text{Snap}_{t,v}^R(\text{enc}(H))) \cong \text{Ctx}_{t,v}(H),$$

and the encoded snapshot is closed under references. The domain premise holds automatically for $\mathcal{J}_D$, $\mathcal{J}_{HO}$ and $\mathcal{J}_A$ and for optional annotations of any depth; for $\mathcal{J}_{BB}$ and $\mathcal{J}_U$ it additionally requires extensionality of $\text{Ctx}_{t,v}(H)$. If also $J_c \subseteq J_d$ for every $c \rightsquigarrow d$ (Policy S), then $\text{Ret}_{t,v} = A_{t,v}$ and the statement is the snapshot commutation of [26] for these images.

Proof. Each row belongs to exactly one component and all rows of a component have the same clocks, so selection keeps all or none of a component's rows. $\text{Ret}_{t,v}$ is $\rightsquigarrow$ -closed by definition, and every $c \in \text{Ret}_{t,v}$ has $t \in L_c$: an asserted c by definition, a retained c because it is reached along $c' \rightsquigarrow c$ from an asserted or retained c' with $t \in L_{c'} \subseteq L_c$. Hence the selected rows reference only selected rows. Atom components are retained whole, so mandatory cardinalities hold. Since enc is a clause-by-clause map, $\text{enc}(\text{Ctx}_{t,v}(H))$ has exactly the selected rows. If $\text{Ctx}_{t,v}(H)$ is in the source domain the untimed roundtrip gives the second equation. For $\mathcal{J}_D, \mathcal{J}_{HO}, \mathcal{J}_A$ the domain conditions are local to atom components, whose ends are retained by closure, so they are inherited from the history. Under Policy S, $A_{t,v}$ is already $\rightsquigarrow$ -closed because $c \rightsquigarrow d$ and $v \in J_c \subseteq J_d$, $t \in L_c \subseteq L_d$ give $d \in A_{t,v}$. $\square$

Remark 1 (Annotation outside the bearer's interval). Let f be a fact valid [2020, 2022] and $a = (m, \text{status}, \text{Disputed})$ an annotation of its membership m asserted from 2025. Policy S forbids this history. Under Policy R the snapshot at $(t, 2025)$ contains a , the membership m and the fact as retained referents; the fact is present in the context but its stored interval shows it is not asserted at 2025. A consumer that must not treat retained rows as claims filters by the stored valid interval.

Remark 2 (Extensional images are fragile under time). The source-domain premise does not follow from well-formed storage for extensional images. Take two edges with equal ends, one valid [0, 5) and one valid [3, 8): the snapshot at $v = 4$ contains both and violates (B1) although each row-local condition holds. Extensionality is a property of each snapshot of the *history*, not of any row, so a time-oblivious image constraint cannot guarantee it. This pushes practical systems toward the identified images $\mathcal{J}_D$ and $\mathcal{J}_{HO}$.

If a role can end independently while its edge anchor survives, endpoint cardinalities can fail and the theorem's premise is violated, so raw row cascade is insufficient to preserve an image. Effective validity computed by intersection on read is a separate policy whose decoding contract needs its own definition.

6.1 Identity across versions

Mandatory fields share their owner's clocks, so an edge whose target is revised becomes a new atom, and nothing above records that the two atoms are one edge, revised. For agent memory, where beliefs are revised, this needs an explicit relation.

Definition 9 (Supersession). A *supersession* is a set of rows $(h(a'), \text{mg}:\text{supersedes}, h(a))$ between atoms of the same class, treated as declared lineage rows in the sense of [26] (their targets must exist but need not be live). It is *admissible* if (V1) each atom supersedes at most one atom and is superseded by at most one, (V2) $\sup L_a < \inf L_{a'}$ whenever a' supersedes a (version lifetimes are ordered and disjoint), and (V3) valid intervals are unconstrained.

(V1) and (V2) make supersession a disjoint union of finite chains; write $[a]$ for the chain of a .

Proposition 9 (Version identity). *Under (V1)–(V2): (i) at every transaction time t each chain has at most one live atom; (ii) the map $a \mapsto [a]$ is well defined and injective on the live atoms of any snapshot; (iii) hence naming the atoms of each snapshot by their chains yields models that can be compared across times, and $[e] = [e']$ holds exactly when e' is a revision of e .*

Proof. (i) Two versions in one chain are ordered by repeated application of (V2), so their lifetimes are disjoint. (ii) Chains partition the atoms by (V1). (iii) follows from (i) and (ii). $\square$

What is not proved: when x is superseded by x' , every live component depending on x must be re-targeted, which is the cascade-and-replay correction of [26]; we do not restate it, and supersession alone does not perform it.

7 Role Walks and Executable Metapaths

Fix an identified directed structure and a candidate $F \subseteq E$. A *simple path* in the metagraph literature [5, 25] is a nonempty edge sequence $e_1, \dots, e_n$ with $x \in V_{e_1}$, $y \in W_{e_n}$ and $W_{e_i} \cap V_{e_{i+1}} \neq \emptyset$. Repetition is allowed, so this is a walk notion.

Proposition 10 (Role-walk equivalence). *Such a path from x to y exists exactly when $h(y)$ is reachable from $h(x)$ by the role expression $(\text{mg:in/mg:out})^+$ in the directed encoding.*

Proof. One step goes backwards from an input anchor to its edge anchor and forwards to an output anchor. Consecutive steps pick an element of each required intersection, and conversely an intersection witness supplies each intermediate step. $\square$

Reading the expression in SPARQL. Write ι for the mapping from rows to RDF 1.2: each row $e = (s, p, o)$ becomes the asserted triple $\langle s \rangle p \langle o \rangle$ together with a reifier resource $\langle e \rangle$ and $\langle e \rangle \text{rdf:reifies} \langle \langle s p o \rangle \rangle$, where $\langle x \rangle = x$ for nodes and literals and $\langle e' \rangle$ is the reifier resource of a row e' . Role rows then have reifier resources as subject and object, and $(\text{mg:in/mg:out})^+$ is an ordinary SPARQL 1.1 property path over those resources. It binds $?x$ to $\langle h(x) \rangle$, not to the anchor node n_x ; the node is recovered by the extra pattern $?x \text{rdf:reifies} \langle (?n \text{rdf:type} ?c) \rangle$. In plain RDF 1.1, where a statement has no resource, the expression is not available, and in RDF-star only quoted triples, not occurrences, appear as terms. In Tiramemsu statement identifiers are usable as nodes, which is an engine extension and not standard property-path semantics.

Definition 10 (Candidate metapath semantics). For $B, C \subseteq X$, F is a *metapath* when

1. every edge of F occurs on a simple path from B to C using only edges of F ;
2. $(\bigcup_{e \in F} V_e) \setminus (\bigcup_{e \in F} W_e) \subseteq B$;
3. $C \subseteq \bigcup_{e \in F} W_e$.

The set conditions follow [25]; we choose the F -restricted reading of the first condition. Empty F is admitted exactly when the conditions hold, in particular only when C is empty.

Theorem 11 (Candidate recognition). *Given B, C, F , recognition under Definition 10 takes expected time $O(R_F + |F| + |B| + |C|)$, where $R_F = \sum_{e \in F} (|V_e| + |W_e|)$, using indexed identities and expected constant-time set operations.*

Proof. Build the incidence graph with an arc $x \rightarrow e$ per input and $e \rightarrow y$ per output. A forward search from B and a reverse search from C mark reached states. An edge lies on a B -to- C walk exactly when its edge state is marked by both: concatenate prefix and suffix witnesses, allowing repetition. The set conditions need one more scan of the role lists. The graph has R_F arcs. $\square$

Do not materialize all input–output pairs of an edge: that can need $|V_e||W_e|$ arcs instead of $|V_e| + |W_e|$.

7.1 Executable metapaths

Definition 10 asks that every edge lie on a walk. It does not ask that the edge can *fire* when it needs all its inputs, as in Gallo et al.’s B-connectivity [9].

Definition 11 (Firing closure). For $B \subseteq X$ and $F \subseteq E$ set $R_0 = B$ and $R_{i+1} = R_i \cup \bigcup \{W_e : e \in F, V_e \subseteq R_i\}$, and let $R_B(F) = \bigcup_i R_i$. An edge *fires* if $V_e \subseteq R_B(F)$. F is an *executable metapath* from B to C if it is a metapath (Definition 10) and every edge of F fires. Let $\text{Walk}_B(F)$ be the set of y such that some edge sequence in F with $V_{e_1} \cap B \neq \emptyset$ ends with $y \in W_{e_n}$.

Example 1 (Walk-metapath that cannot execute). Let $B = \{b\}$, $C = \{c\}$, $e_1 = (\{b, a\}, \{d\})$, $e_2 = (\{d\}, \{a, c\})$. Both edges lie on a walk from b to c , the pure inputs are $\{b\}$, and c is produced, so $\{e_1, e_2\}$ is a metapath. But $R_B(F) = \{b\}$: e_1 needs a , produced only by e_2 , which needs d , produced only by e_1 . Neither edge fires.

Theorem 12 (Executable recognition). *Whether F is an executable metapath from B to C is decidable in expected time $O(R_F + |F| + |B| + |C|)$. Moreover (a) $R_B(F) \subseteq B \cup \text{Walk}_B(F)$; (b) if $|V_e| = 1$ for all $e \in F$ the two sets are equal; (c) in general the inclusion is strict (Example 1).*

Proof. Keep for each edge a counter of its inputs not yet reached, and a queue seeded with B . Popping a vertex decrements the counters of the edges it feeds; an edge whose counter reaches zero fires and enqueues its outputs. Every arc is touched once, giving $O(R_F + |B|)$, and the metapath conditions are checked by Theorem 11. (a) By induction on the stage at which $y \in W_e$ is added: pick $x' \in V_e \subseteq R_i$, which is nonempty. If $x' \in B$ the one-edge sequence e witnesses y . Otherwise x' has a witnessing sequence ending in an edge with x' among its outputs, and appending e extends it. (b) If $y \in W_{e_n}$ has a witnessing sequence, $V_{e_1} \cap B \neq \emptyset$ and $|V_{e_1}| = 1$ give $V_{e_1} \subseteq B$, so e_1 fires; and $V_{e_{i+1}}$ being a singleton meeting W_{e_i} is contained in $W_{e_i} \subseteq R_B(F)$, so each edge fires in turn. $\square$

Theorem 13 (Coincidence). *Let F be a metapath from B to C . Call $e \rightarrow e'$ in the dependency graph of F when $W_e \cap V_{e'} \neq \emptyset$ (loops included). If this graph is acyclic, or if $|V_e| = 1$ for every $e \in F$, then every edge of F fires, so F is an executable metapath. In general the converse direction fails by Example 1, whose dependency graph has the cycle $e_1 \rightarrow e_2 \rightarrow e_1$.*

Proof. Acyclic case: process edges in a topological order of the dependency graph. For e and $v \in V_e$, either $v \in W_{e'}$ for some $e' \in F$, so $e' \rightarrow e$ and e' fired earlier, giving $v \in R_B(F)$; or v lies in $\bigcup V \setminus \bigcup W \subseteq B$ by condition 2. So $V_e \subseteq R_B(F)$. Singleton case: by condition 1 every e is the last edge of a prefix of a walk from B within F , and the argument of Theorem 12(b) shows it fires. $\square$

B-connectivity is Horn-clause forward chaining; a property path follows walks and cannot require that all of V_e be reached, and we do not prove an inexpressibility result. The engine does not provide this recognizer as an operator; the checker of Section 10 implements it and tests the claims above.

Dominance and projection. Edge dominance means that no proper subset of F is a metapath for B, C ; input dominance means that no proper subset of B reaches C by any metapath in the source edge universe; dominance is their conjunction [25]. The recognition bound does not give a bound for finding or enumerating dominant metapaths. In the identified parallel-edge extension, k equal edges from $\{b\}$ to $\{c\}$ give $2^k - 1$ metapaths.

An external projection P transfers by $\tilde{P} = \text{enc} \circ P \circ \text{dec}$ on an exact input image, and its structural output decodes to $P(M)$ by the roundtrip result. Basu–Blanning projection and transitivity-preserving projection have different contracts [5, 25]. Adding metapath containers and `summarises` relations yields an enriched `Row4` store whose *structural reduct*, obtained by

deleting rows outside the image vocabulary together with the rows that reference them, is the projection image. This deletion is a projection, not a normalization. Several supporting metapaths must be recorded separately, and invalidation needs an explicit maintenance policy.

8 Depth, Rank and Its Consumers

Reference rank earns its place only if something consumes it. Two operations do: loading a referentially complete store, and retracting a row under dependent cascade.

Proposition 14 (Bulk-load strata). *Let S be a finite acyclic store. A partition $S = S_1 \uplus \dots \uplus S_k$ in which every row's statement endpoints lie in earlier parts exists if and only if $k \geq 1 + \max_e \text{rk}(e)$. The rank partition $\Lambda_0, \dots, \Lambda_{\max}$ attains the minimum.*

Proof. The rank partition qualifies because each reference strictly lowers rank. Conversely, a row of rank m is the top of a reference chain $e_m \rightarrow \dots \rightarrow e_0$ of $m + 1$ rows, and each must lie in a strictly later part than the one it references, so at least $m + 1$ parts are needed. $\square$

So a bulk load that must never insert a row before its referents needs exactly $1 + \max \text{rk}$ batches. Anchored encodings need at most two batches however deeply atoms are nested, because rank counts reference depth and not nesting depth; a compact binary encoding needs $1 + \delta_{\max}$ (Theorem 18); an Row_4 store with annotation chains of length k needs $k + 1$ or more.

Proposition 15 (Retraction-cascade depth). *Under dependent-row cascade, retracting e removes exactly $\uparrow e = \{d : d \rightarrow^* e\}$. Every $d \in \uparrow e \setminus \{e\}$ has $\text{rk}(d) > \text{rk}(e)$, and the cascade, expanded in rounds from e , terminates after at most $\max \text{rk} - \text{rk}(e)$ rounds after the first.*

Proof. Dependent-row cascade deletes rows with a reference to a deleted row, so the deleted set is the reverse reachability closure. A row d at reference distance j from e has a path $d \rightarrow \dots \rightarrow e$ of length j , so $\text{rk}(d) \geq \text{rk}(e) + j$ by the rank recurrence, and $j \leq \max \text{rk} - \text{rk}(e)$. $\square$

In an anchored encoding, retracting a structural row (rank one) therefore touches only annotations of it and annotations of those, in at most $\max \text{rk} - 1$ further rounds, and retracting an anchor takes its incident structural rows in the first round. These are statements about passes over the reference graph. We make no claim about wall-clock cost.

Theorem 16 (Anchored structural ceiling). *In Row_3 , anchors and ground facts have rank zero and every other row has rank one. The same holds of every row of each exact image of Section 4. In Row_4 , rank counts reference dependency depth and may exceed one.*

Proof. Anchors and ground facts have no statement endpoints; all remaining Row_3 shapes reference only anchors or ground facts. Direction and ground-valued attribute rows in $\mathcal{J}_A$ also reference only anchors. Each has at least one reference, so its rank is one. The recurrence applies to the unrestricted Row_4 references. $\square$

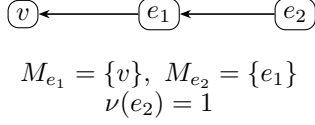
The ceiling holds because atoms are represented as typing rows rather than as nodes; if atoms were nodes, structural rows would all have rank zero, and the choice of anchors is what places structure at rank one. It is a property of this representation.

For a recursive-member structure define $\nu(e) = 0$ if all members are fundamental vertices and $\nu(e) = 1 + \max_{a \in M_e \cap E} \nu(a)$ otherwise.

Proposition 17 (Structural nesting). *For $G \in \mathcal{J}_U$, $\nu(e)$ is the longest member-role path from $h(e)$, minus one.*

Proof. Vertices have role-path length zero and nonempty edges have length $1 + \max_{a \in M_e} \pi(a)$. Induction on the acyclic member relation gives $\pi(e) = \nu(e) + 1$. $\square$

Decoded member relation



Statement references

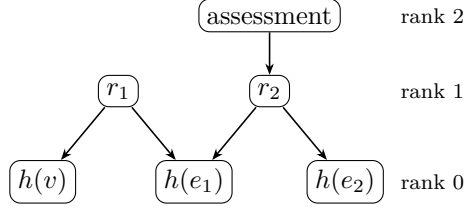


Figure 1: Member roles $r_1 = (h(e_1), \text{mg:member}, h(v))$ and $r_2 = (h(e_2), \text{mg:member}, h(e_1))$ encode nested edges at reference rank one. An optional assessment of r_2 adds rank two, hence a third load stratum. The left arrows are decoded member incidence; the right arrows are references between statement identifiers.

Arbitrary membership graphs may be cyclic, so longest-walk depth is not asserted for all containers: finite reachability and finite nesting depth differ.

Definition 12 (Compact binary encoding). For an acyclic identified higher-order structure with singleton ends y, z , retain ground Vertex anchors and encode each edge by one row $\kappa(h(e)) = (h(y), p_e, h(z))$ with a fixed node predicate p_e . Set $\delta(v) = 0$ and $\delta(e) = 1 + \max(\delta(y), \delta(z))$.

Theorem 18 (Compact depth). $\text{rk}(h(x)) = \delta(x)$ for every atom in the compact encoding.

Proof. Induction on endpoint-dependency depth: vertices are ground anchors and each edge references exactly its two encoded endpoints (one distinct referent if they agree). $\square$

This restates the rank recurrence; its value is that, together with Proposition 14, it prices the compact encoding at $1 + \delta_{\max}$ load strata against two for the anchored one. Decoding compact stores needs a separate convention to distinguish vertex anchors from edge rows, for example reserving `rdf:type` for anchors.

Proposition 19 (Exact row counts). Let $R = \sum_{e \in E} (|V_e| + |W_e|)$ and $U = \sum_{e \in E} |M_e|$. Directed and higher-order encodings have $|X| + |E| + R$ rows; the member encoding has $|X| + |E| + U$ rows. For Definition 8, with $Q = \sum_c |F_c|$ and $T = \sum_x |\text{attr}(x)|$, the encoding has $|A| + 3|E \cup M E| + Q + T$ rows. Compact binary encoding has $|X| + |E|$ rows.

Proof. Count the disjoint anchor and field clauses of each encoding. $\square$

These are row counts, not storage sizes: they ignore index and page overhead. With indexed atom lookup and expected constant-time hash-set operations, encoding and decoding take expected time linear in these counts. Not every image constraint is checkable row by row: acyclicity needs traversal and extensionality needs comparison of grouped role sets.

Rank is not an epistemic type. A confidence row $(h(x), \text{confidence}, 0.7)$ has rank one and a confidence on a membership has rank two; both may express epistemic content. An approval represented as an anchored event can have all structural rows at rank one, while an approval as a property of a fact and an audit of that property give ranks one and two. Rank is an encoding-dependent depth measure whose consumers are loading and retraction, not a test for reflection.

9 A Provenance-Aware Memory Example

Episodic and semantic memory have distinct roles in cognitive accounts [34], and reconstruction and scripts motivate schema-guided recall [3, 31]. Contemporary agent-memory systems include temporal graph architectures and combined semantic/episodic world models [2, 29]. Our example concerns representation of their kinds of information, not a comparison of retrieval performance.

Example 2 (Membership with its own evidence). Let a be a Vertex anchor marked as an episode container, f a ground fact about Sarah’s job and m its episode membership. The store contains:

Identifier	Content	Rank
a	$(n_a, \text{rdf:type}, \text{mg:Vertex})$	0
u	$(a, \text{mg:container}, \text{true})$	1
f	$(\text{sarah}, \text{announced}, \text{newJob})$	0
m	$(f, \text{sys:inGraph}, a)$	1
o	$(m, \text{status}, \text{Observed})$	2
c	$(m, \text{confidence}, 0.7)$	2
p	$(c, \text{derivedFrom}, \text{transcript7})$	3

A confidence $(f, \text{confidence}, 0.9)$ about the job fact itself would have rank one, so confidence in a fact and in its contextual membership are different claims. Loading this store takes four strata (Proposition 14); retracting m removes o , c and p in at most two rounds (Proposition 15) and leaves f and a . A later dispute of m , valid in a year after f ’s interval, is admissible under Policy R of Section 6 and not under Policy S. The store is an enriched Row_4 representation, not an exact attributed image.

Schemas can be marked containers whose slots are anchored atoms; slot fillings and specialization are typed relations, and scripts add precedence relations between scene anchors. These shapes do not enforce slot cardinality, satisfiability, causality or temporal consistency of a script. An episode may contain observed, inferred, disputed or recalled material, and membership alone does not imply observation; explicit status predicates and derivation links record these distinctions. A constructive retrieval procedure may emit default-based claims with **Inferred** status linked to supporting default occurrences, and retracting one default invalidates only a support depending on it; we do not implement or evaluate it here. The six semantic categories proposed in [27] can be retained as type annotations; rank must not be used as a substitute for them.

10 Realization and Validation

10.1 Implementation boundary

Tiramemsu is a Rust/SQLite statement-identity database with SPARQL and Cypher interfaces [26]. Its repository recipe `metagraph_containers_nesting_and_fold` demonstrates cyclic container reachability, grouping without fact copying, an edge used as a graph name, and retraction of memberships without deleting contained ground facts. One example query is:

```
PREFIX v: <urn:v:>
SELECT ?s WHERE {
  ?patient v:enrolledIn ?trial ~ ?edge .
  GRAPH ?edge { ?s ?p ?o }
}
```

The statement-binding syntax is an engine extension. The engine does not enforce the image constraints at write time and does not provide the metapath, dominance or projection routines as library operators. The validators and checker below operate on finite stores separately from engine execution.

10.2 Executable validators

The image constraints are expressible as executable validators over a row view of the store: cardinality and class constraints as SHACL core shapes, and acyclicity (`?e mg:member+ ?e`)

and extensionality (comparison of grouped role sets) as SPARQL. The files, in `validators/`, are tested for agreement with the Python image validators: on 80,665 store comparisons the SHACL and SPARQL verdicts and the Python verdicts never differ. The comparisons are exhaustive for the roundtrip sources under each family, for all 65,536 role-subset stores (2,401 accepted) and for the recursive member sets, and sampled (seeded) for attributed stores and for single-row mutations of valid stores. The row view projects each anchor row $(n, \text{rdf:type}, C)$ to a triple on its identifier, because the toolchain used (rdflib, pyshacl) lacks RDF 1.2 reifier patterns. This shows the images can be stated as shapes and queries over a standard representation and that these agree with the checker on the tested sets; it does not make the engine enforce them, and agreement on a finite set is not a proof.

10.3 Reproduction protocol

The accompanying standard-library Python artifact, `check_encodings.py` and its extension module, retains source encoders, independently scanned image validators and decoders, an incidence-search recognizer, an edge-sequence enumeration oracle, a firing-closure implementation with a naive fixpoint oracle, and the property tests listed below. Run:

```
python3 check_encodings.py --output validation-results.json
```

Exhaustive enumeration uses fixed bounds and no sampling. It covers directed and higher-order roundtrips; all subsets of 16 possible role rows over two vertices and two edges (including self-end roles); recursive member structures with cycle and extensionality rejection; four-class attributed structures with empty and overlapping fragments; metapath queries over all B, C, F in a two-vertex, at-most-two-edge domain; and the necessity witnesses of Table 2. Accepted candidate stores equal the two-edge source encodings exactly: the 2,411 higher-order source roundtrips are the 1 zero-edge, 9 one-edge and 2,401 two-edge sources, and the 2,401 accepted candidates of Table 3 are that last group, seen from the store side.

Seeded randomized tests (fixed seed, up to eight vertices and eight edges for roundtrips and up to three edges for the sequence oracle) check, for each family, roundtrip on random valid models; that any randomly mutated store the validator accepts re-encodes to itself up to isomorphism; the strata and cascade propositions on random Row_4 stores; firing closure against a naive fixpoint and Theorems 12 and 13 on random structures; and Theorem 8 under both policies on random histories. The oracle that enumerates edge sequences is exponential, so its randomized domain is small. These tests exercise sizes beyond the exhaustive bounds but remain finite samples, not proof.

Mechanized proof. The directory `lean/` holds a Lean 4 development (core library only) of the directed roundtrip, parametrized over $\mathcal{J}_D$, $\mathcal{J}_{HO}$ and the extensional variants. It proves $\text{dec}(\text{enc}(M)) \simeq M$ for valid sources, $\text{enc}(\text{dec}(G)) = G$ as row sets (and up to permutation) for image stores, membership of $\text{enc}(M)$ in the image, the row count of Proposition 19, and the rank bound of Theorem 16 for these stores, with no `sorry`, `axiom` or `native_decide`. It fixes canonical identifiers, so it proves the clause bijection of Lemma 3 rather than the quantification over renamings of Definition 3; it models image membership as a scan-then-check predicate, close to the validity of the decoded model, and it does not cover the recursive, attributed or temporal results.

Passing finite checks supports the hand proofs and detects small counterexamples; it is not proof for unbounded structures. The checker reports no performance measurements.

11 Discussion and Limitations

The practical consequence is a separation of concerns. A uniform carrier can retain identity and annotate incidence, while the exact source image still needs explicit integrity enforcement, and

Check family	Cases
Exhaustive: directed roundtrips (identified edges)	91
Exhaustive: Basu–Blanning roundtrips (extensional)	82
Exhaustive: higher-order roundtrips ($1 + 9 + 2,401$ sources)	2,411
Exhaustive: higher-order candidate stores	65,536
Accepted / rejected candidates ($2,401 = \text{two-edge sources}$)	2,401 / 63,135
Exhaustive: recursive candidates / accepted roundtrips	49 / 30
Exhaustive: four-class attributed roundtrips	65,536
Exhaustive: metapath query/oracle comparisons	5,488
Exhaustive: compact depth configurations	14,400
Exhaustive: dependency-closed temporal selections	20
Targeted malformed-image rejections	9
Necessity witnesses (one per constraint)	18
Sampled: random valid models, roundtrip	7,500
Sampled: mutated stores accepted / rejected	3,622 / 11,378
Sampled: strata and cascade checks (random Row_4)	1,500
Exhaustive: firing-closure cases (3 vertices, ≤ 2 edges)	620,992
Sampled: firing-closure cases with sequence oracle	4,000
Metapaths with coincidence antecedent / strict gap	51,176 / 50,143
Sampled: snapshot points, Policy R / Policy S	24,000 / 24,000
Sampled: version chains, violations rejected	3,000 / 2,246

Table 3: Counts produced by the retained checkers. A case is a model, store or query configuration, not an independent theorem. Bounds and the exhaustive/sampled distinction are recorded in `validation-results.json`.

typed rows alone do not establish edge completeness, extensionality or well-founded nesting. Row-level deletion and source-level deletion differ when one atom is represented by several rows. Under time, extensional images are fragile (Remark 2), which favours the identified images.

Scope. The source restrictions matter: directed ends and recursive edges are nonempty, recursive incidence is well founded and attributed values are ground. Annotating presentations that identify atoms by all their fields, allow other endpoint classes or attach graph-valued attributes need different signatures and images. We do not claim that identified triples are a minimal universal storage primitive, that inclusion of row languages is a semantic expressiveness separation, that these encodings are faster, smaller or more expressive than relational or graph alternatives, or that reference rank classifies epistemic content. A systems evaluation should compare matched incidence and reification baselines [17], include index and storage overhead and separate role-heavy reads from integrity-maintaining updates; a memory-system evaluation should measure retrieval and provenance handling with ablations of identity, annotated membership and time. Neither is part of this paper.

Evidence. Validation is finite enumeration, seeded randomized testing, executable validators and a partial Lean proof; there is no external review. The mechanized proof covers one family under canonical naming. Version propagation under revision, effective validity on read and the general dominant-metapath problem are not treated. Full-text comparison with the three recent metagraph works remains necessary before any priority claim about operations or granulation. The accompanying source audit records access limits and the reference metadata checked in this revision; the inherited bibliography has not been wholly audited.

12 Conclusion

Identified triples admit invertible encodings of explicitly specified finite metagraph structures when exact image constraints distinguish identity, incidence and containment, and each constraint is necessary. Under time, a policy that retains referents of asserted rows lets annotations live outside their bearer’s interval, and a supersession relation records identity across versions. Executable metapaths can be recognized in linear incidence size and coincide with walk metapaths when the dependency graph is acyclic or tails are singletons. Reference rank prices loading and retraction. Enforcing these contracts efficiently in an engine and measuring their value for agent memory remain implementation and evaluation tasks.

Availability. The source, PDF, checkers, validators, Lean proof and validation results accompany Tiramemsu at <https://github.com/Volland/tiramemsu>, in `paper/metagraphs-from-identified-triples/`. Tiramemsu uses MIT and Apache-2.0 licenses.

References

- [1] Renzo Angles, Aidan Hogan, Ora Lassila, Carlos Rojas, Daniel Schwabe, Pedro Szekely, and Domagoj Vrgoč. Multilayer graphs: a unified data model for graph databases. In *Proc. 5th ACM SIGMOD Joint Int. Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 11:1–11:6, 2022. doi: 10.1145/3534540.3534696.
- [2] Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Andrey Kravchenko, Mikhail Burtsev, and Evgeny Burnaev. Arigraph: Learning knowledge graph world models with episodic memory for llm agents, 2024. URL <https://arxiv.org/abs/2407.04363>.
- [3] Frederic C. Bartlett. *Remembering: A Study in Experimental and Social Psychology*. Cambridge University Press, 1932.
- [4] Amit Basu and Robert W. Blanning. Metagraphs: A tool for modeling decision support systems. *Management Science*, 40(12):1579–1600, 1994. doi: 10.1287/mnsc.40.12.1579.
- [5] Amit Basu and Robert W. Blanning. *Metagraphs and Their Applications*, volume 15 of *Integrated Series in Information Systems*. Springer, 2007. doi: 10.1007/978-0-387-37234-1.
- [6] Valeriy M. Chernenkiy, Yuriy E. Gapanyuk, Georgiy I. Revunkov, Yuriy T. Kaganov, Yuriy S. Fedorenko, and Svetlana V. Minakova. Using metagraph approach for complex domains description. In *Selected Papers of DAMDID/RCDL 2017*, volume 2022 of *CEUR Workshop Proceedings*, pages 342–349, 2017. URL <https://ceur-ws.org/Vol-2022/paper52.pdf>.
- [7] Valeriy M. Chernenkiy, Yuriy E. Gapanyuk, Yuriy T. Kaganov, Ivan V. Dunin, Maxim A. Lyaskovsky, and Vadim S. Larionov. Storing metagraph model in relational, document-oriented, and graph databases. In *Selected Papers of DAMDID/RCDL 2018*, volume 2277 of *CEUR Workshop Proceedings*, pages 82–89, 2018. URL <https://ceur-ws.org/Vol-2277/paper17.pdf>.
- [8] Mikhail Galkin, Priyansh Trivedi, Gaurav Maheshwari, Ricardo Usbeck, and Jens Lehmann. Message passing for hyper-relational knowledge graphs. In *Proc. 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7346–7359, 2020. doi: 10.18653/v1/2020.emnlp-main.596.
- [9] Giorgio Gallo, Giustino Longo, Stefano Pallottino, and Sang Nguyen. Directed hypergraphs and applications. *Discrete Applied Mathematics*, 42(2–3):177–201, 1993. doi: 10.1016/0166-218X(93)90045-P.

- [10] Yuriy Gapanyuk. The development of the metagraph data and knowledge model. In *Selected Contributions to IMSC-2021*, volume 2965 of *CEUR Workshop Proceedings*, pages 1–7, 2021. URL <https://ceur-ws.org/Vol-2965/paper01.pdf>.
- [11] Ewout Gelling, George Fletcher, and Michael Schmidt. Bridging graph data models: RDF, RDF-star, and property graphs as directed acyclic graphs [extended abstract], 2023. URL <https://arxiv.org/abs/2304.13097>.
- [12] Ewout Gelling, George Fletcher, and Michael Schmidt. Statement graphs: Unifying the graph data model landscape. In *Database Systems for Advanced Applications (DASFAA 2024)*, LNCS, pages 364–376. Springer, 2024. doi: 10.1007/978-981-97-5575-2_27.
- [13] Ben Goertzel. Folding and unfolding on metagraphs, 2020. URL <https://arxiv.org/abs/2012.01759>.
- [14] Ben Goertzel, Cassio Pennachin, and Nil Geisweiller. The OpenCog framework. In *Engineering General Intelligence, Part 2*, pages 3–29. Atlantis Press, 2014. doi: 10.2991/978-94-6239-030-0_1.
- [15] David Harel. On visual formalisms. *Communications of the ACM*, 31(5):514–530, 1988. doi: 10.1145/42411.42414.
- [16] Olaf Hartig, Pierre-Antoine Champin, Andy Seaborne, and Gregg Kellogg. RDF 1.2 concepts and abstract data model. W3C Candidate Recommendation Snapshot, 2026. URL <https://www.w3.org/TR/2026/CR-rdf12-concepts-20260407/>. 7 April 2026; not a final Recommendation.
- [17] Daniel Hernández, Aidan Hogan, and Markus Krötzsch. Reifying RDF: What works well with Wikidata? In *Proc. 11th International Workshop on Scalable Semantic Web Knowledge Base Systems (SSWS)*, volume 1457 of *CEUR Workshop Proceedings*, pages 32–47, 2015. URL https://ceur-ws.org/Vol-1457/SSWS2015_paper3.pdf.
- [18] Borislav Iordanov. Hypergraphdb: A generalized graph database. In *WAIM 2010 Workshops*, volume 6185 of *LNCS*, pages 25–36. Springer, 2010. doi: 10.1007/978-3-642-16720-1_3.
- [19] Cliff Joslyn and Kathleen Nowak. Ubergraphs: A definition of a recursive hypergraph structure, 2017. URL <https://arxiv.org/abs/1704.05547>. PNNL-26402.
- [20] Irina Karabulatova, Stepan Vinnikov, Anatoly Nardid, and Yuriy Gapanyuk. The metagraph transformation algorithm based on incidence and nesting representation. DAMDID 2025 conference programme, short-paper abstract, 2025. URL https://damdid2025.frccsc.ru/en/conference_program.html. Conference precursor; distinguished from the 2026 journal article.
- [21] Irina S. Karabulatova, Stepan S. Vinnikov, Anatoliy N. Nardid, and Yuriy E. Gapanyuk. The metagraph transformation algorithm based on incidence and nesting representation. *Pattern Recognition and Image Analysis*, 36(2):616–628, 2026. doi: 10.1134/S1054661826700422.
- [22] Krishna Kulkarni and Jan-Eike Michels. Temporal features in SQL:2011. *ACM SIGMOD Record*, 41(3):34–43, 2012. doi: 10.1145/2380776.2380786.
- [23] Robin Milner. *The Space and Motion of Communicating Agents*. Cambridge University Press, 2009.
- [24] Natasha Noy and Alan Rector. Defining n-ary relations on the semantic web. W3C Working Group Note, 2006. URL <https://www.w3.org/TR/swbp-n-aryRelations/>. 12 April 2006.

- [25] Eric Parsonage, Matthew Roughan, and Hung X. Nguyen. Transitivity preserving projection in directed hypergraphs, 2025. URL <https://arxiv.org/abs/2509.04543>.
- [26] Volodymyr Pavlyshyn. Layered bitemporal graphs: Statement identity, two clocks, and metagraphs as tags. Preprint, revision 4, 3 October 2026, 2026. URL <https://doi.org/10.5281/zenodo.23139228>.
- [27] Volodymyr Pavlyshyn. *Metagraph for AI Agents: Metagraphs for Agentic AI: Beyond Triples, Beyond Hypergraphs*. Leanpub, 2026. URL <https://leanpub.com/metagraphforaiagents>. Self-published e-book; prior informal work by the author.
- [28] Alexandra Poulovassilis and Mark Levene. A nested-graph model for the representation and manipulation of complex objects. *ACM Transactions on Information Systems*, 12(1): 35–68, 1994. doi: 10.1145/174608.174610.
- [29] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
- [30] Sepehr Sadoughi, Nikolay Yakovets, and George Fletcher. Meta-property graphs: Extending property graphs with metadata awareness and reification, 2024. URL <https://arxiv.org/abs/2410.13813>.
- [31] Roger C. Schank and Robert P. Abelson. *Scripts, Plans, Goals and Understanding: An Inquiry into Human Knowledge Structures*. Lawrence Erlbaum Associates, 1977.
- [32] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, San Francisco, 1999. ISBN 1-55860-436-7.
- [33] Valery Tarassov, Yuriy Kaganov, and Yuriy Gapanyuk. The metagraph model for complex networks: Definition, calculus, and granulation issues. In *Artificial Intelligence (RCAI 2021)*, volume 12948 of *Lecture Notes in Computer Science*, pages 135–151. Springer, 2021. doi: 10.1007/978-3-030-86855-0_10. URL https://link.springer.com/chapter/10.1007/978-3-030-86855-0_10.
- [34] Endel Tulving. Episodic and semantic memory. In Endel Tulving and Wayne Donaldson, editors, *Organization of Memory*, pages 381–403. Academic Press, 1972.
- [35] TypeDB. TypeQL data model: entities, relations, attributes and role players. Product documentation, 2026. URL <https://typedb.com/docs/typeql-reference/data-model/>. Accessed 5 October 2026.
- [36] Stepan Vinnikov, Anatoly Nardid, and Yuriy Gapanyuk. Metagraph operations using bigraph representation. In Panos Pardalos, Eduard Babkin, Nikolay Zolotykh, and Sergey Stupnikov, editors, *Data Analytics and Management in Data Intensive Domains*, volume 2641 of *Communications in Computer and Information Science*, pages 32–48. Springer, 2025. doi: 10.1007/978-3-032-03997-2_3. URL https://link.springer.com/chapter/10.1007/978-3-032-03997-2_3.
- [37] Domagoj Vrgoč, Carlos Rojas, Renzo Angles, Marcelo Arenas, Diego Arroyuelo, Carlos Buil-Aranda, Aidan Hogan, Gonzalo Navarro, Cristian Riveros, and Juan Romero. Millenniumdb: An open-source graph database system. *Data Intelligence*, 5(3):560–610, 2023. doi: 10.1162/dint_a_00229.